

EventForce Management System

Salesforce Implementation Report

Overview

The EventForce Management System is a cloud-based CRM solution engineered on Salesforce to digitize, centralize, and automate end-to-end operations for Event Planner Solutions. By replacing fragmented, manual spreadsheets, the CRM establishes an integrated platform handling client bookings, venue reservations, vendor service coordination, automated timeline alerts, and post-event feedback collection.

The system leverages declarative tools (Lightning App Builder, Record-Triggered Flows, Approval Processes, and Validation Rules) alongside programmatic enhancements (Apex Classes, Apex Triggers, and Batch/Schedulable Apex) to provide real-time operational visibility and eliminate human scheduling errors.

Objectives

Centralized Data Management: Unify disparate operational records (Clients, Events, Venues, Vendors, Feedback) into a single relational schema to ensure seamless cross-functional collaboration.

60% Reduction in Manual Processes: Automate multi-step approval workflows, timeline-based client notifications, venue status toggles, and daily past-event archival.

40% Improvement in Booking Efficiency: Eliminate scheduling conflicts and venue double-bookings via Apex trigger validation.

Enhanced Stakeholder Engagement: Send automated 3-day reminder notices to clients and coordinators to keep all parties aligned.

Data-Driven Decision Making: Deliver real-time operational insights into event distribution, monthly budgets, and vendor performance via custom reports and dynamic dashboards.

Strict Data Governance and Security: Establish private baseline access across sensitive event data while leveraging role hierarchies, permission sets, and targeted sharing rules to guarantee appropriate visibility for administrators, coordinators, and external partners.

Standardized Financial Forecasting: Automate accurate event budget calculations across various event categories through dynamic formula logic, removing guesswork and manual cost entry errors.

Optimized Resource Allocation: Maintain real-time tracking of venue availability states and service partner assignments through custom junction relationships, preventing resource underutilization and overlapping vendor contracts.

Closed-Loop Quality Control: Automate client feedback collection and contextual validation to directly evaluate vendor reliability and overall event success, driving continuous service enhancement.

Operational Continuity and Scalability: Ensure zero system disruption during high-volume booking cycles by handling routine maintenance through scheduled background batch processes and scalable cloud infrastructure.

PHASE 1: REQUIREMENT ANALYSIS & PLANNING

Functional Scope: Design custom data architecture, enforce data hygiene via validation rules and lookup filters, automate operational updates via Salesforce Flows and Apex, establish cancellation governance, enforce granular role-based security, and implement executive reporting.

Data Governance & Integrity Constraints:

Enforced clean data entry at inception through contextual lookup filters specifically restricting feedback event lookups strictly to events associated with the selected client.

Designed automated budget formula logic to calculate base financial projections without manual calculation.

Formulated validation rules to safeguard essential customer contact data against improper formatting.

Stakeholder Mapping:

Clients: Request event services, receive schedule alerts, and submit feedback.

Event Coordinators: Manage booking schedules, track vendor deliverables, and initiate cancellation approvals.

Vendors: Deliver assigned operational services (catering, photography, decor, DJ/audio).

Venue Managers: Track physical space reservations and availability.

Event Administrators: Maintain system integrity, monitor batch jobs, and manage user security.

PHASE 2: BACKEND DEVELOPMENT & CONFIGURATIONS

Data Schema & Relationships

Custom Objects & Schema:

Client__c: Tracks client identity and regional demographics (Client Name, Email, Phone, Address, Country, City).

Venue__c: Manages physical event spaces (Venue Name, Address, Location, Capacity, Availability Status [Available, Reserved, Not Available]).

Vendor__c: Catalogs external service partners (Vendor Name, Email, Phone, Service Type [Multi-Select], Status).

Event__c: Core transaction object (Event Name, Event Date, Event Type, Event Status, Event Budget).

Feedback__c: Captures client satisfaction (Feedback ID [Auto-Number: F-{0000}], Rating, Comments).

EventVendor__c: Junction object establishing a Many-to-Many relationship between Event__c and Vendor__c via dual Master-Detail relationships.

Relational Model:

Event__c → Client__c (Lookup)

Event__c → Venue__c (Lookup)

Feedback__c → Event__c (Lookup)

Feedback__c → Client__c (Lookup)

EventVendor__c → Event__c (Master-Detail)

EventVendor__c → Vendor__c (Master-Detail)

The screenshot shows the Salesforce Object Manager interface. At the top, there's a navigation bar with 'Setup', 'Home', and 'Object Manager' tabs. Below this, a header section contains the 'Object Manager' title, a 'Quick Find' search bar, and buttons for 'Schema Builder' and 'Create'. The main area displays a table of objects with columns for object name, API name, type, and creation date. The table lists various standard and custom objects, including Engagement Channel Type, Entitlement, Event, Event Vendor, Feedback, Finance Balance Snapshot, Finance Transaction, and Fulfillment Order.

Object Name	API Name	Type	Created	Status
Engagement Channel Type	EngagementChannelType	Standard Object		
Engagement Channel Work Type	EngagementChannelWorkType	Standard Object		
Entitlement	Entitlement	Standard Object		
Entitlement Contact	EntitlementContact	Standard Object		
Event	Event__c	Custom Object	9/30/2026	✓
Event	Event	Standard Object		
Event Vendor	Event_Vendor__c	Custom Object	9/28/2026	✓
Feedback	Feedback__c	Custom Object	9/28/2026	✓
Finance Balance Snapshot	FinanceBalanceSnapshot	Standard Object		
Finance Transaction	FinanceTransaction	Standard Object		
Fulfillment Order	FulfillmentOrder	Standard Object		

Formula Fields;

Event Budget Formula (Event_Budget__c - Currency): Standardizes dynamic budget estimation using a conditional CASE expression based on Event_Type__c:

sql

CASE(

TEXT(Event_Type__c),

"Wedding", 50000,

"Corporate", 30000,

"Birthday", 10000,

"Anniversary", 20000,

"Festival", 60000,

"Concert", 40000,

"Other", 15000,

0

)

Feedback Contextual Lookup Filter: Applied on Feedback__c.Event__c so that selecting a Client restricts visible Events to records where Event__c.Client__c == Feedback__c.Client__c.

Validation Rules

Email Format Enforcement: Implemented on `Client\_\_c` using a regular expression to enforce valid email formats upon insert/update:

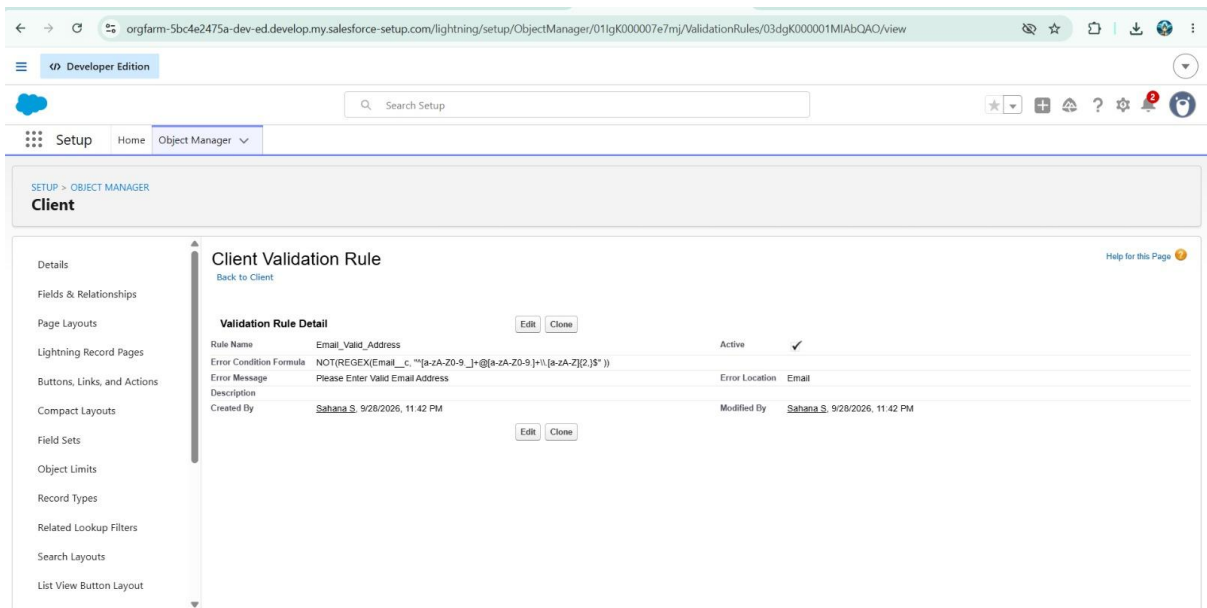
Rule Name: Email_Valid_Address

Formula: NOT(REGEX(Email__c, "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}\$"))

Error Message:"Please Enter Valid Email Address"

The screenshot shows the Salesforce Setup interface. The left sidebar contains a navigation menu with options like Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Search Layouts, and List View Button Layout. The main content area is titled 'Validation Rules' and shows a table with one rule named 'Email_Valid_Address'. The table has columns for Rule Name, Error Location, Error Message, Active status, and Modified By. The rule is active and was modified by Logesh NJ on 9/29/2026 at 11:13 PM. The error message is 'Please Enter Valid Email Address'.

RULE NAME	ERROR LOCATION	ERROR MESSAGE	ACTIVE	MODIFIED BY
Email_Valid_Address	Email	Please Enter Valid Email Address	✓	Logesh NJ, 9/29/2026, 11:13 PM



Business Automation: Flows & Approval Processes

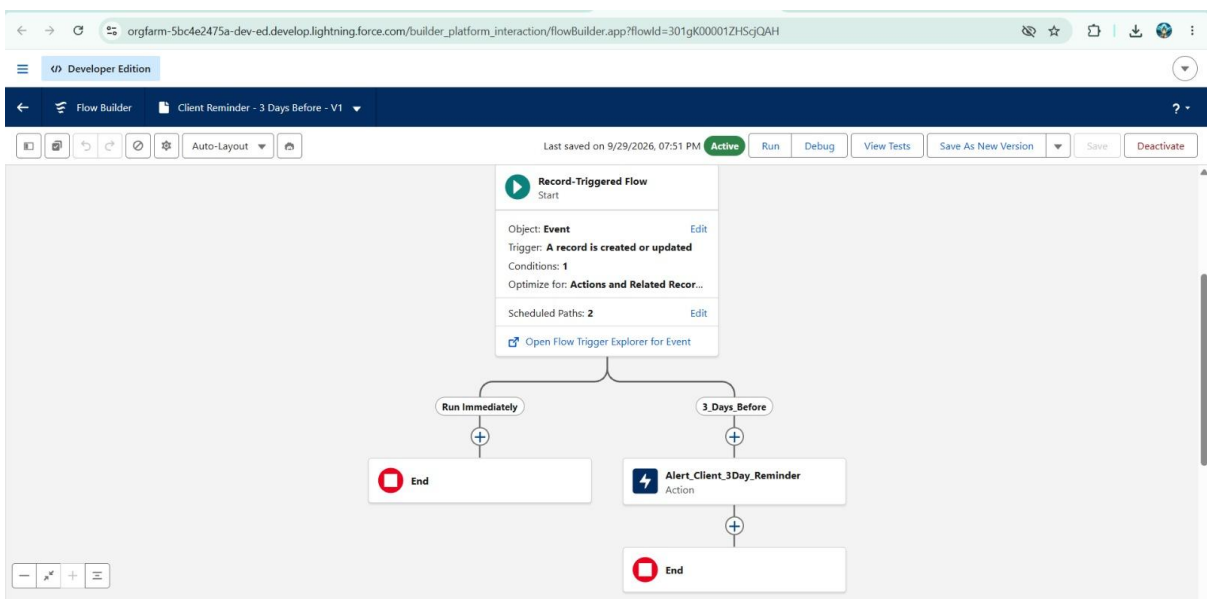
Record-Triggered Flow (Client Reminder - 3 Days Before):

Trigger: Record created/updated with Event_Status__c = Confirmed.

Optimization: Actions and Related Records.

Scheduled Path: Configured for 3 Days Before the Event_Date__c field.

Action: Issues an automated email alert (Send Email) pulling plain-text merge templates to the assigned Client, CC-ing the Event Owner.



Event Cancellation Approval Process (Event_Cancellation_Process):

Entry Criteria: Event__c.Event_Status__c EQUALS "Pending Cancellation".

Submission Action: Locks the record and issues email alerts using Classic Email Templates to coordinators.

Approval Action: Updates Event_Status__c to Cancelled and notifies both the Event Owner and Client.

Rejection Action: Resets Event_Status__c to Rejected (or reverts to Confirmed) and alerts the Event Owner.

The screenshot shows the Salesforce Setup interface for the 'Event_Cancellation_Process' approval process. The page is titled 'Approval Processes' and includes a search bar. The left sidebar shows the 'Setup' menu with 'Approval Processes' selected. The main content area displays the 'Process Definition Detail' for the 'Event_Cancellation_Process'. The process is active and managed by the 'Manager of Record Submitter'. The entry criteria is 'Event: Event_Status EQUALS Pending Cancellation'. The record editability is 'Administrator OR Current Approver'. The approval assignment email template is 'Event_Cancellation_Request_Notification'. The initial submission actions are 'Record Lock' and 'Email Alert'. The process was created by 'Sahana S.' on 9/29/2026 at 1:53 AM and modified by 'Sahana S.' on 9/29/2026 at 2:49 AM.

Process Name	Unique Name	Description	Entry Criteria	Record Editability	Approval Assignment Email Template	Initial Submitters	Created By	Modified By
Event_Cancellation_Process	Event_Cancellation_Process		Event: Event_Status EQUALS Pending Cancellation	Administrator OR Current Approver	Event_Cancellation_Request_Notification	Event Owner	Sahana S. 9/29/2026, 1:53 AM	Sahana S. 9/29/2026, 2:49 AM

Action	Type	Description
Record Lock		Lock the record from being edited
Email Alert		Alert - Event Cancellation Request

Apex Triggers & Scheduled Batch Jobs

Venue Status Automation: An after-insert/after-update trigger (EventTrigger13) invokes VenueStatusHelper.updateVenueStatus() to automatically toggle Venue__c.Availability_Status__c to Reserved when an event is confirmed, and reset it to Available if cancelled.

PROGRAM:

```
public with sharing class VenueStatusHelper {  
    public static void updateVenueStatus(List<Event__c> events) {  
        List<Venue__c> venues = new List<Venue__c>();  
  
        for (Event__c ev : events) {  
            if (ev.Venue__c != null) {  
                if (ev.Event_Status__c == 'Confirmed') {  
                    venues.add(new Venue__c(  
                        Id = ev.Venue__c,  
                        Availability_Status__c = 'Reserved'  
                    ));  
                } else if (ev.Event_Status__c == 'Canceled') {  
                    venues.add(new Venue__c(  
                        Id = ev.Venue__c,  
                        Availability_Status__c = 'Available'  
                    ));  
                }  
            }  
        }  
  
        if (!venues.isEmpty()) {  
            update venues;  
        }  
    }  
}
```

Double-Booking Prevention Trigger (PreventDoubleBooking): A before-insert/before-update trigger inspects existing database reservations and blocks updates if another event is already confirmed for that venue on the requested date:

PROGRAM:

```
trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();

    for (Event__c ev : Trigger.new) {
        if (ev.Venue__c != null) {
            venueIds.add(ev.Venue__c);
        }
    }

    Map<Id, List<Event__c>> venueEventMap = new Map<Id,
List<Event__c>>();
    for (Event__c e : [SELECT Id, Venue__c, Event_Date__c
                        FROM Event__c
                        WHERE Venue__c IN :venueIds]) {
        if (!venueEventMap.containsKey(e.Venue__c)) {
            venueEventMap.put(e.Venue__c, new List<Event__c>());
        }
        venueEventMap.get(e.Venue__c).add(e);
    }

    for (Event__c ev : Trigger.new) {
        if (ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)) {
            for (Event__c existing : venueEventMap.get(ev.Venue__c)) {
                if (existing.Event_Date__c == ev.Event_Date__c && existing.Id !=
ev.Id) {
                    ev.addError('This Venue is already booked on this date.');
```

```

    }
}
}
}

```

Daily Event Lifecycle Maintenance: A batch class (BatchCompleteEvents) queries past events (Event_Date__c < TODAY and status not completed) and updates them to Completed. This is invoked by a schedulable wrapper (ScheduleCompleteEvents) registered through the Salesforce Setup scheduler to run nightly at 8:00 PM.

The screenshot shows the Salesforce Developer Console in a Brave browser window. The address bar displays the URL: orgfarm-37b6669331-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage. The console has tabs for 'BatchCompleteEvents.apex' and 'VenueStatusHelper.apex', with 'BatchCompleteEvents.apex' selected. The code editor shows the following Apex code:

```

1 global class ScheduleCompleteEvents implements Schedulable {
2
3     global void execute(SchedulableContext sc) {
4         Database.executeBatch(new BatchCompleteEvents(), 200);
5     }
6 }

```

Below the code editor, there are tabs for 'Logs', 'Tests', 'Checkpoints', 'Query Editor', 'View State', 'Progress', and 'Problems'. The 'Logs' tab is active, showing a table with columns: User, Application, Operation, Time, Status, Read, and Size. The table is currently empty. At the bottom of the console, there is a filter button and a link to filter the log list. The Windows taskbar at the bottom shows the date and time as 9:01 AM on 10/2/2026, along with system icons for weather (28°C Sunny), search, and various application icons.

Developer Console - Brave

orglarm-37b6669331-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage

File Edit Debug Test Workspace Help

BatchCompleteEvents.apxc ScheduleCompleteEvents.apxc VenuesStatusHelper.apxc

Code Coverage: None API Version: 67 Go To

```
1 global class BatchCompleteEvents implements Database.Batchable<sObject> {
2
3     global Database.QueryLocator start(Database.BatchableContext bc) {
4         return Database.getQueryLocator(
5             'SELECT Id, Event_Date__c, Event_Status__c FROM Event__c WHERE Event_Date__c < TODAY AND Event_Status__c != \'Completed\''
6         );
7     }
8
9     global void execute(Database.BatchableContext bc, List<Event__c> scope) {
10         for (Event__c ev : scope) {
11             ev.Event_Status__c = 'Completed';
12         }
13         update scope;
14     }
15
16     global void finish(Database.BatchableContext bc) {
17         System.debug('Past events marked as Completed');
18     }
19 }
```

Logs Tests Checkpoints Query Editor View State Progress Problems

User	Application	Operation	Time	Status	Read	Size
------	-------------	-----------	------	--------	------	------

Filter Click here to filter the log list

28°C Sunny

Search

ENG IN 83% 9:01 AM 10/2/2026

PHASE 3: UI/UX CUSTOMIZATION & ANALYTICS

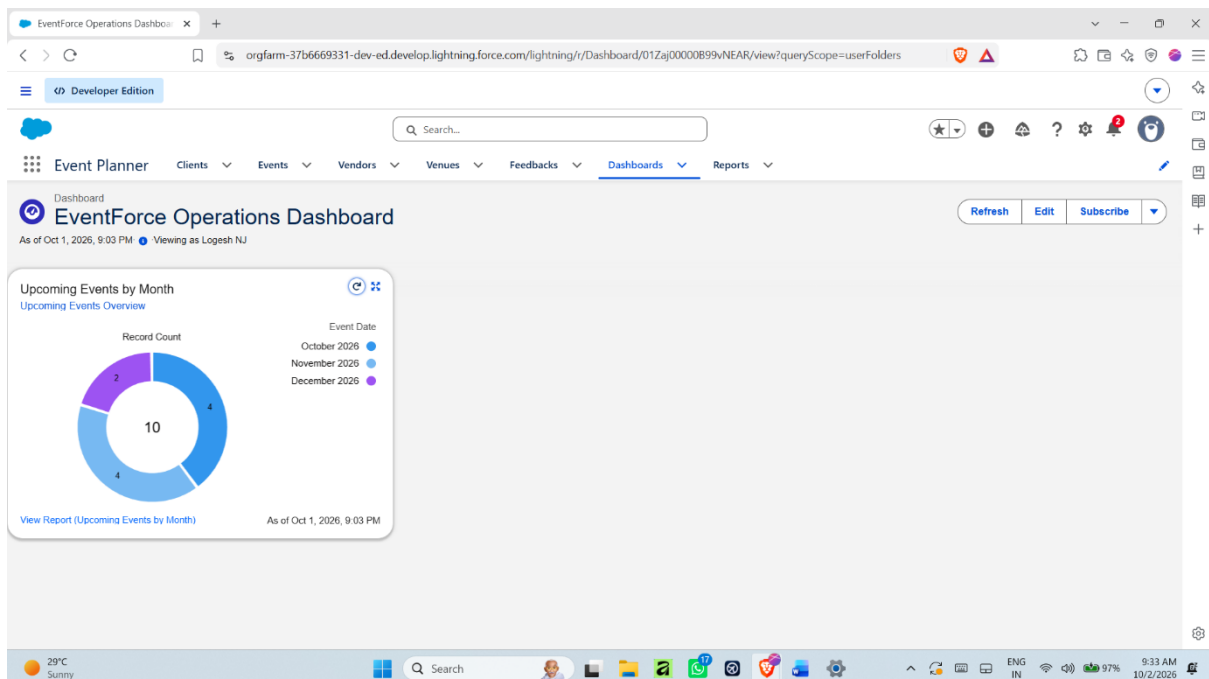
Custom Lightning App (Event Planner): Configured via Lightning App Manager with custom branding, an intuitive navigation bar, and accessible tabs: Events, Clients, Vendors, Venues, Feedbacks, Reports, and Dashboards.

Operational Reporting:

Report Folder: EventForce Report, shared with View access for the Event Coordinator and Vendor Manager roles.

Summary Report (Upcoming Events by Month): Uses grouping by calendar month on Event_Date__c, filters out canceled events, displays record counts, and sums total budget metrics (SUM(Event_Budget__c)).

Executive Dashboard (EventForce Operations Dashboard): Features donut and horizontal bar charts highlighting upcoming event volumes and cumulative revenue per calendar month.



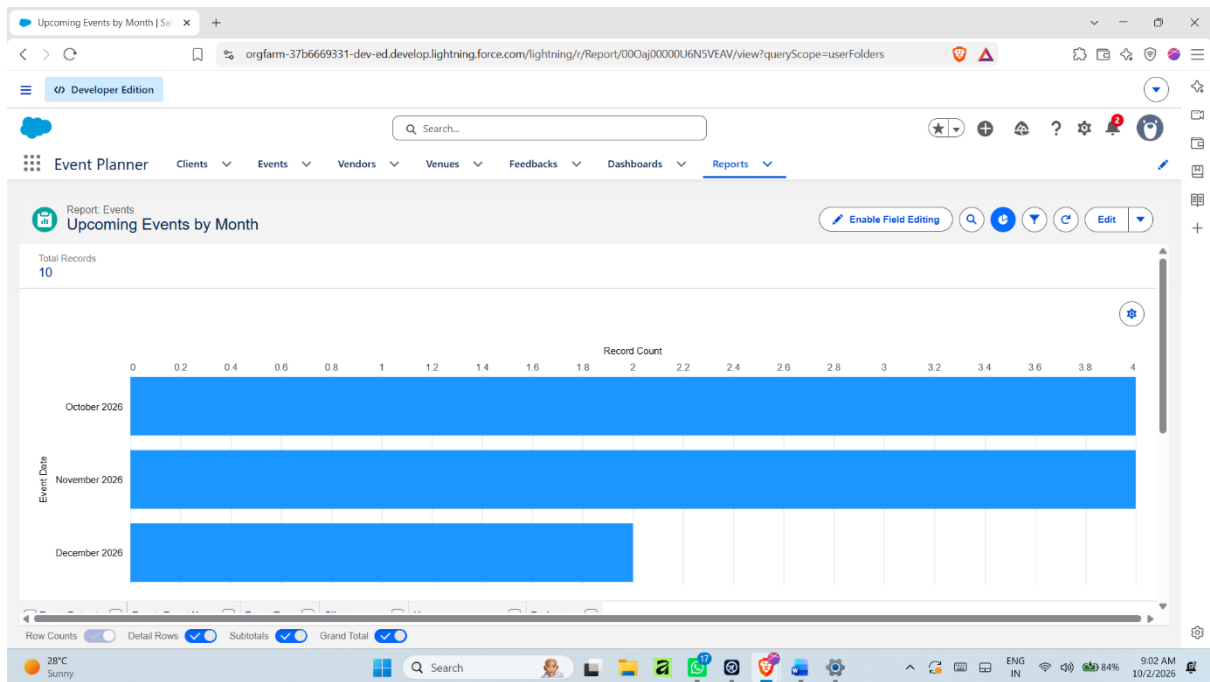
Upcoming Events by Month | Salesforce Developer Edition

Report: Events
Upcoming Events by Month

Enable Field Editing

Event Date	Event: Event Name	Event Type	Client	Venue	Budget
October 2026 (4)	Upcoming Event 1	Wedding	Client 1 Corp	Grand Venue Hall 1	\$50,000.00
	Upcoming Event 2	Corporate	Client 2 Corp	Grand Venue Hall 2	\$30,000.00
	Upcoming Event 3	Birthday	Client 3 Corp	Grand Venue Hall 3	\$10,000.00
	Upcoming Event 4	Anniversary	Client 4 Corp	Grand Venue Hall 4	\$20,000.00
Subtotal					
November 2026 (4)	Upcoming Event 5	Festival	Client 5 Corp	Grand Venue Hall 5	\$80,000.00
	Upcoming Event 6	Concert	Client 6 Corp	Grand Venue Hall 6	\$40,000.00
	Upcoming Event 7	Other	Client 7 Corp	Grand Venue Hall 7	\$15,000.00
	Upcoming Event 8	Wedding	Client 8 Corp	Grand Venue Hall 8	\$50,000.00
Subtotal					
December 2026 (2)	Upcoming Event 9	Corporate	Client 9 Corp	Grand Venue Hall 9	\$30,000.00
	Upcoming Event 10	Birthday	Client 10 Corp	Grand Venue Hall 10	\$10,000.00
Subtotal					
Total (10)					

Row Counts: ☐ Detail Rows: ☒ Subtotals: ☒ Grand Total: ☒



PHASE 4: DATA MIGRATION, TESTING & SECURITY SETUP

Data Migration: Baseline data for clients, venues, and vendors was cleaned, validated for duplicate constraints, and imported into Developer Edition using the Salesforce Data Import Wizard.

Organization-Wide Defaults (OWD):

Event__c: Set to Private (internal and external) to restrict baseline record visibility exclusively to record owners and higher-level roles.

Client__c, Vendor__c, Venue__c: Set to Public Read/Write.

Role Hierarchy: Scaled under executive leadership (CEO → Event Admin → child roles: Event Coordinator, Vendor Manager, Client) to mirror the corporate chain of command.

Profile-Level Access & Password Policies:

Event Admin: Cloned from System Administrator for full customization and data governance.

Event Coordinator: Base Standard Platform User with CRED on Events, Read/Edit on Clients, Read-Only on Vendors/Venues, and Create/Read on Feedback.

Vendor Manager: Base Standard Platform User with CRED on Vendors and Read-Only on Events, Venues, and Feedback.

Client Profile: Restricted Read-Only visibility to personal Events and Feedback submission.

Permission Sets: Implemented the Feedback Manager permission set granting elevated Read/Edit/Create/Delete object rights to targeted coordinators without changing their base profile.

Criteria & Sharing Rules: Deployed rule Event_Sharing_For_Vendors, automatically granting Read Only access to events owned by members of the Event Coordinator role to users assigned to the Vendor Manager role

The screenshot shows a Salesforce Lightning page in 'Developer Edition' mode. A modal form titled 'Information' is open, displaying fields for a new client record. The fields include Client Name (Test client), Email (abc), Phone (1234567890), Address (chennai), Country (India), and City (Chennai). The Owner is Sahana S. A red error message box is overlaid on the form, stating 'We hit a snag. Review the following fields: Email'. The background shows a list of recently viewed clients.

Client Name	Owner
Test client	Sahana S

Fields in the form:

- Client Name: Test client
- Email: abc (Error: We hit a snag. Review the following fields: Email)
- Phone: 1234567890
- Address: chennai
- Country: India
- City: Chennai

The screenshot shows the Salesforce Setup page in 'Developer Edition' mode. The 'Client Layout' configuration is displayed, showing a list of fields and a sample layout. The fields include Client Name, Country, Owner, Address, Created By, Phone, City, and Email. The sample layout shows a 'Client Sample' section with a 'Highlights Panel' and 'Quick Actions in the Salesforce Classic Publisher'.

Client Layout configuration:

- Fields: Client Name, Country, Owner, Address, Created By, Phone, City, Email
- Sample Layout: Client Sample, Highlights Panel, Quick Actions in the Salesforce Classic Publisher

PHASE 5: DEPLOYMENT STRATEGY, MAINTENANCE & DOCUMENTATION

Deployment Strategy

In this project lifecycle, all configurations, custom metadata, and code components were implemented and validated inside a standalone Salesforce Developer Edition Org. While academic and sandbox developer orgs operate independently of production environments, enterprise implementations utilize the following deployment strategy:

Environment Architecture: Development → Sandbox / Scratch Org → UAT (User Acceptance Testing) → Staging → Production.

Migration Mechanism: Deployment of metadata packages (Custom Objects, Layouts, Flows, Triggers, Profiles) using Salesforce CLI (SFDX), change sets, or Git-based CI/CD pipelines (e.g., GitHub Actions / Jenkins).

Pre-Deployment Checklist: Execute all test methods to confirm an Apex unit test coverage above the mandatory 75% threshold, validate Flow trigger ordering, and ensure profile/permission set parity.

Maintenance, Monitoring & Troubleshooting

Automated Log Monitoring: Track apex execution failures, flow run errors, and email delivery exceptions via Setup Apex Jobs, Paused and Failed Flow Interviews, and Email Logs.

Data Hygiene Maintenance: Run weekly duplicate check routines and verify that the nightly `ScheduleCompleteEvents` scheduled batch job consistently updates past event records.

Access Control Audits: Regularly audit profile assignments, permission sets, and sharing rules to maintain least-privilege access and avoid privilege escalation.

Documentation Summary

The project documentation serves as the single source of truth for all stakeholders, administrators, and developers. It captures the underlying business rationale, full data dictionary, entity-relationship models, process automation logic, role-based access models, and operational runbooks for continued platform evolution.

Conclusion & Project Outcomes

The EventForce Management System implementation highlights the power of Salesforce as an extensible, enterprise-grade application engine. By balancing declarative tools (Flows, Approvals, Page Layouts) with programmatic customizations (Triggers, Schedulable Apex), the platform achieves end-to-end automation while maintaining strict security via fine-grained OWD and role hierarchies.

The resulting CRM successfully consolidates event booking operations, eliminates manual scheduling conflicts, provides real-time budget forecasting, and ensures a seamless experience for clients, coordinators, and vendors alike.